

3.2 – Systemd Services

Making Your Python Script Run Like a Real Application on Raspberry Pi

EdgeAI Engineering Bootcamp — Analog Data

What You Will Learn

By the end of this session you will be able to:

- Explain what systemd is and why it exists
 - Write a complete `.service` unit file for a Python script
 - Use `Restart=always` and `RestartSec=5` for automatic crash recovery
 - Load environment variables securely using `EnvironmentFile=` and a `.env` file
 - Start, stop, restart, enable, and check the status of any service
 - Stream and filter live logs using `journalctl`
 - Troubleshoot a service that fails to start
-

What You Will Build

By the end of this session, you will have a working Edge AI inference service running on your Raspberry Pi that:

- Starts automatically every time the Pi boots
 - Restarts itself within 5 seconds if it ever crashes
 - Loads all configuration (model path, broker IP, API keys) from a `.env` file
 - Writes structured logs that you can read and filter live from any terminal
-

Part 1 — The Problem with Running Scripts Manually

Let's say you have written a Python script that runs your AI model on a Raspberry Pi.

You SSH into the Pi, type:

```
python3 inference.py
```

It works perfectly. But then:

Situation	What happens
You close the terminal	The script dies immediately
The Pi restarts after a power cut	The script never starts again
The script crashes due to a bug	It stays dead until you manually fix it
You disconnect from SSH	Every process you started dies

This is the core problem. Scripts that are run manually in a terminal only live as long as that terminal session is open. The moment the session ends — whether you close it, lose network, or the Pi reboots — the script is gone.

What a real production application needs:

- Start automatically when the Pi boots — no human intervention needed
- If it crashes, restart itself within a few seconds
- Run completely in the background — no terminal required
- Be controllable with simple commands: start, stop, restart, status

This is exactly what **systemd** provides.

Part 2 — What Is systemd?

systemd is the system and service manager built into Raspberry Pi OS and virtually every modern Linux distribution. It is the very first process that starts when the Pi boots and it stays running until the Pi shuts down.

systemd manages everything that runs on the system — network connections, Bluetooth, SSH server, and now your Python scripts.

A **service** is any program that systemd manages. You describe how to run your program in a file called a **unit file** (a small text file that ends in `.service`). Once you give systemd that file, it takes care of starting, stopping, restarting, and monitoring your program.

Analogy: Think of systemd as a factory operations manager. You hand the manager a job card (the `.service` file) that says: "Run this worker (Python script). If the worker quits unexpectedly, hire a replacement within 5 seconds. Give the worker these tools (environment variables) when they start." The manager handles everything. You go home. The factory keeps running.

Part 3 — The **systemctl** Command

systemctl is the tool you use to interact with systemd from the terminal. Learn these commands — you will use them every single day.

```
# Reload systemd after creating or editing a service file
sudo systemctl daemon-reload
```

```
# Start a service right now
sudo systemctl start edgeai

# Stop a running service
sudo systemctl stop edgeai

# Restart a service (stop then start)
sudo systemctl restart edgeai

# Check the current status of a service
sudo systemctl status edgeai

# Enable a service – auto-start when Pi boots
sudo systemctl enable edgeai

# Disable a service – stop it from auto-starting at boot
sudo systemctl disable edgeai
```

Important: `start` and `enable` do different things.

- `start` → runs the service **right now**
- `enable` → marks the service to **automatically start on every future boot**

You almost always want to do both. Starting without enabling means the service dies on the next reboot.

Other useful `systemctl` commands:

```
# See all currently running services
systemctl list-units --type=service --state=running

# View the service file exactly as systemd reads it
systemctl cat edgeai

# Check if a service is currently active (useful inside scripts)
systemctl is-active edgeai      # prints: active or inactive

# Check if a service is enabled for boot (useful inside scripts)
systemctl is-enabled edgeai     # prints: enabled or disabled
```

Part 4 — Anatomy of a `.service` Unit File

A unit file is a plain text file divided into three sections. Each section starts with its name inside square brackets.

```
[Unit]           ← General information: description, startup order
[Service]       ← How to run the program: command, user, restart behaviour
```

[Install] ← When to run this service during the boot sequence

All service files live in:

```
/etc/systemd/system/
```

The file name becomes the service name. A file called `edgeai.service` creates a service called `edgeai`.

Part 5 — The [Unit] Section

```
[Unit]
Description=Edge AI Inference Service
After=network.target
StartLimitIntervalSec=60
StartLimitBurst=5
```

Description=

A short, human-readable name for your service. This text appears in `systemctl status` output and in the journal logs. Make it descriptive enough to understand at a glance.

```
Description=Edge AI Inference Service    # Good
Description=myapp                        # Not useful
```

After=

Tells systemd the startup order — which system milestones must be reached before your service starts.

```
After=network.target
```

Your inference service likely needs to connect to an MQTT broker, download a config, or talk to a cloud API. None of that works before the network is up. `network.target` is a built-in systemd milestone that means "networking is available."

Common `After=` values:

Value	Meaning
<code>network.target</code>	Network interfaces are up
<code>network-online.target</code>	Network is up AND internet is reachable
<code>time-sync.target</code>	System clock has been synchronised via NTP
<code>multi-user.target</code>	Full system boot is complete

You can list more than one:

```
After=network.target time-sync.target
```

StartLimitIntervalSec= and StartLimitBurst=

```
StartLimitIntervalSec=60
StartLimitBurst=5
```

If your script has a bug that causes it to crash immediately on startup, systemd with `Restart=always` would restart it thousands of times per minute — wasting CPU and thrashing the SD card with logs.

These two lines together mean: **"If the service crashes and restarts more than 5 times within 60 seconds, stop trying to restart it."**

This protects your Pi from runaway crash loops. When the limit is hit, `systemctl status` will show `failed` and you will need to fix the root cause and then run `sudo systemctl reset-failed edgeai` before starting it again.

Part 6 — The [Service] Section

This is the most important section. It tells systemd exactly how to run your program.

```
[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/edgeai
EnvironmentFile=/home/pi/edgeai/.env
ExecStartPre=/bin/bash -c 'test -f ${MODEL_PATH}'
ExecStart=/usr/bin/python3 /home/pi/edgeai/inference.py
ExecStop=/bin/kill -SIGTERM $MAINPID
Restart=always
RestartSec=5
TimeoutStopSec=10
StandardOutput=journal
StandardError=journal
SyslogIdentifier=edgeai
```

Let's go through every line.

Type=simple

Tells systemd what kind of process this is.

Type	When to use
<code>simple</code>	Your program runs in the foreground and keeps running. Use this for all Python scripts.
<code>oneshot</code>	Program runs once and exits. Good for setup or cleanup scripts.
<code>forking</code>	Program forks itself into the background. Used by some older software — rare.

User=pi

Which Linux user account to run the script as.

```
User=pi
```

Running as `pi` (your normal user account) gives the script access to your files, camera, GPIO, and I2C without needing root privileges.

Never use `User=root` unless absolutely necessary. Running as root means a bug in your Python code has permission to accidentally damage the entire operating system.

WorkingDirectory=

Sets the current working directory before running the script. This is the equivalent of doing `cd /home/pi/edgeai` before running the command.

```
WorkingDirectory=/home/pi/edgeai
```

This matters because your Python script might use relative file paths like:

```
open("./models/model.tflite")    # relative path
```

Without `WorkingDirectory`, `systemd` starts from `/` (root of the filesystem) and this path would fail. With `WorkingDirectory=/home/pi/edgeai`, relative paths work as expected.

EnvironmentFile=

Loads environment variables from a file before starting your script.

```
EnvironmentFile=/home/pi/edgeai/.env
```

This is covered in full detail in Part 7.

ExecStartPre=

A command to run **before** `ExecStart` . If this command fails, the service will not start. Use it to check preconditions.

```
ExecStartPre=/bin/bash -c 'test -f ${MODEL_PATH}'
```

This checks whether the model file exists before trying to start the inference script. If the model file is missing, `ExecStartPre` fails, the service does not start, and you get a clear error in the logs — instead of starting Python only to have it crash with a `FileNotFoundError` three seconds later.

ExecStart=

The command to run your program. **Always use full absolute paths** — never relative paths or command names without their full path.

```
# WRONG — systemd does not search PATH like a terminal does
ExecStart=python3 inference.py

# RIGHT — full path to both python3 and the script
ExecStart=/usr/bin/python3 /home/pi/edgeai/inference.py
```

To find the full path of python3 on your Pi:

```
which python3
# /usr/bin/python3
```

If you use a virtual environment, point to the python inside it:

```
ExecStart=/home/pi/edgeai/venv/bin/python3 /home/pi/edgeai/inference.py
```

Restart=always

This is the line that makes your service self-healing.

```
Restart=always
```

With `Restart=always` , if your script exits for **any reason** — exception, segfault, killed by OOM, anything — systemd automatically restarts it.

Value	Behaviour
no	Never restart. Service stays dead if it exits. (Default)
always	Always restart, no matter why it stopped

Value	Behaviour
<code>on-failure</code>	Restart only if the script exited with a non-zero exit code (error). If you manually run <code>systemctl stop</code> , it will NOT restart.
<code>on-abnormal</code>	Restart on crash or timeout, but not on clean normal exit

For a production Edge AI service you want `Restart=always` — it should be running continuously no matter what.

RestartSec=5

How many seconds to wait before restarting after the service exits.

```
RestartSec=5
```

Without this, systemd restarts immediately. This is fine for crashes that happen mid-run. But if your script crashes in the first two seconds of startup (model not loaded yet, network not ready, wrong config), a zero-second restart delay means systemd tries to restart it hundreds of times per minute — which is why `StartLimitBurst` also exists.

`RestartSec=5` gives the system five seconds to recover from whatever caused the crash before trying again.

ExecStop=

The command to run when you issue `systemctl stop`. By default systemd sends `SIGTERM` to the process, which is usually enough. You can make it explicit:

```
ExecStop=/bin/kill -SIGTERM $MAINPID
```

`$MAINPID` is a special variable systemd fills in with the process ID of your running script.

TimeoutStopSec=

How long systemd waits for your script to shut down cleanly after sending `SIGTERM`, before force-killing it with `SIGKILL`.

```
TimeoutStopSec=10
```

This gives your Python script 10 seconds to catch the signal, finish writing any data, close the camera handle, and exit cleanly. After 10 seconds, systemd force-kills it.

StandardOutput=journal and StandardError=journal

```
StandardOutput=journal
StandardError=journal
```

These tell systemd where to send the output of your script:

- `StandardOutput=journal` → everything your script prints to stdout (`print()` , `logging.info()`) goes to the systemd journal
- `StandardError=journal` → error messages and Python tracebacks go to the same journal

Without these, your script's output goes nowhere — you will have no way to see what it is doing or why it crashed.

SyslogIdentifier=

```
SyslogIdentifier=edgeai
```

The label that appears in the journal next to every log line from this service. Without this, the label defaults to the process name (usually `python3`) — which is not useful when you have multiple Python services running. Setting it to a meaningful name like `edgeai` makes filtering logs much easier.

Part 7 — The [Install] Section

```
[Install]
WantedBy=multi-user.target
```

WantedBy=multi-user.target

This tells systemd **at which point during the boot sequence** to start your service.

`multi-user.target` is the stage where the Pi has fully booted, all system services are running, and the system is ready for normal use. This is the correct target for almost every Pi application.

This line only takes effect when you run `sudo systemctl enable edgeai` . Under the hood, `enable` creates a symbolic link inside `/etc/systemd/system/multi-user.target.wants/` that causes systemd to start your service whenever it reaches `multi-user.target` during boot.

Part 8 — EnvironmentFile and .env — Loading Configuration Securely

The problem with hardcoding values

Look at this Python script:

```
MQTT_BROKER = "192.168.1.100"
API_KEY = "sk_live_abc123verysecretkey"
MODEL_PATH = "/home/pi/models/v2/model.tflite"
```

These values are hardcoded directly in the source code. Problems with this:

- Push the code to GitHub → API keys are exposed publicly
- Deploy to 20 devices with different broker IPs → you need 20 different script versions
- Upgrade the model to v3 → you need to edit and re-deploy the source code
- Change the broker IP → you need to edit code, not just config

The correct approach: store all configuration in a separate `.env` file, never commit it to version control, and load it into your service at runtime using `EnvironmentFile=`.

Step 1 — Create the project folder structure

```
/home/pi/edgeai/
├── inference.py      ← your Python script
├── .env              ← configuration (never commit this)
├── .gitignore        ← tells Git to ignore .env
└── models/
    └── model.tflite  ← your model file
```

Step 2 — Create the `.env` file

```
nano /home/pi/edgeai/.env
```

Format: one `KEY=value` pair per line. No spaces around `=`. No quotes needed.

```
# /home/pi/edgeai/.env
# Edge AI Service Configuration

MODEL_PATH=/home/pi/edgeai/models/model.tflite
CAMERA_ID=0
CONFIDENCE_THRESHOLD=0.75

MQTT_BROKER=192.168.1.100
MQTT_PORT=1883
MQTT_TOPIC=sensors/frames

API_KEY=your_secret_api_key_here
API_ENDPOINT=https://api.example.com/infer

LOG_LEVEL=INFO
```

Lines starting with `#` are comments — they are ignored.

Step 3 — Protect the `.env` file

The `.env` file contains secrets. Set its permissions so only your user can read it:

```
chmod 600 /home/pi/edgeai/.env
```

Verify the permissions:

```
ls -la /home/pi/edgeai/.env
# -rw----- 1 pi pi 312 Jul 17 18:30 /home/pi/edgeai/.env
```

`-rw-----` means only the owner (`pi`) can read and write it. No one else on the system can read your API keys.

Step 4 — Add `.env` to `.gitignore`

```
echo ".env" >> /home/pi/edgeai/.gitignore
```

This tells Git to never track the `.env` file. Your secrets will never accidentally end up in version control.

Step 5 — Reference `EnvironmentFile=` in the service file

```
[Service]
EnvironmentFile=/home/pi/edgeai/.env
ExecStart=/usr/bin/python3 /home/pi/edgeai/inference.py
```

When systemd starts your service, it reads every line in the `.env` file and sets those values as environment variables **before** running your Python script. Your script then reads them via `os.environ`.

Alternative — `Environment=` for single values: If you only need to set one or two values and don't need a file, you can set them directly in the service file:

```
Environment="LOG_LEVEL=INFO"
Environment="CAMERA_ID=0"
```

Use `EnvironmentFile=` when you have multiple values or any secrets. Use `Environment=` only for one-off non-sensitive overrides.

Step 6 — Read values in Python using `os.environ`

```
import os

MODEL_PATH = os.environ.get("MODEL_PATH", "/home/pi/models/model.tflite")
CAMERA_ID = int(os.environ.get("CAMERA_ID", "0"))
CONFIDENCE = float(os.environ.get("CONFIDENCE_THRESHOLD", "0.5"))
MQTT_BROKER = os.environ.get("MQTT_BROKER", "localhost")
MQTT_PORT = int(os.environ.get("MQTT_PORT", "1883"))
API_KEY = os.environ.get("API_KEY", "")
LOG_LEVEL = os.environ.get("LOG_LEVEL", "INFO")
```

How `os.environ.get("KEY", "default")` works:

- Look for an environment variable named `KEY`
- If it is found → use that value
- If it is NOT found → use the `"default"` value as a fallback

The fallback value is important. During local development when you run the script manually (without systemd loading the `.env` file), the fallback keeps the script working with sensible defaults. When running as a service, systemd loads the `.env` and all the real values are used.

Part 9 — The Complete Python Script

Here is a production-ready Python inference script that uses all the patterns above:

```
#!/usr/bin/env python3
"""
Edge AI Inference Service
Analog Data – EdgeAI Engineering Bootcamp

Designed to run as a systemd service. Reads all config from
environment variables loaded from .env by systemd.
"""

import os
import sys
import time
import signal
import logging

# -----
# Load configuration from environment
# -----

MODEL_PATH = os.environ.get("MODEL_PATH", "/home/pi/edgeai/models/model.tflite")
CAMERA_ID = int(os.environ.get("CAMERA_ID", "0"))
CONFIDENCE = float(os.environ.get("CONFIDENCE_THRESHOLD", "0.5"))
MQTT_BROKER = os.environ.get("MQTT_BROKER", "localhost")
MQTT_PORT = int(os.environ.get("MQTT_PORT", "1883"))
LOG_LEVEL = os.environ.get("LOG_LEVEL", "INFO")

# -----
# Set up logging
```

```

# -----
logging.basicConfig(
    level=getattr(logging, LOG_LEVEL.upper(), logging.INFO),
    format="%(asctime)s %(levelname)-8s %(name)s %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
log = logging.getLogger("edgeai")

# -----
# Graceful shutdown handler
# -----
# systemd sends SIGTERM when you run: systemctl stop edgeai
# We catch it here so the script can clean up before exiting.
def handle_shutdown(signal, frame):
    log.info("Shutdown signal received (SIGTERM). Stopping gracefully...")
    # close camera, MQTT client, file handles etc. here
    log.info("Cleanup complete. Goodbye.")
    sys.exit(0)

signal.signal(signal.SIGTERM, handle_shutdown)
signal.signal(signal.SIGINT, handle_shutdown) # also handles Ctrl+C during dev

# -----
# Startup checks
# -----
def run_startup_checks():
    log.info("Running startup checks...")

    if not os.path.isfile(MODEL_PATH):
        log.critical(f"Model file not found: {MODEL_PATH}")
        log.critical("Cannot start without model. Exiting.")
        sys.exit(1) # non-zero exit code triggers Restart=always

    log.info(f"Model found           : {MODEL_PATH}")
    log.info(f"Camera ID           : {CAMERA_ID}")
    log.info(f"Confidence thresh : {CONFIDENCE}")
    log.info(f"MQTT broker        : {MQTT_BROKER}:{MQTT_PORT}")
    log.info("All checks passed. Starting inference loop.")

# -----
# Main inference loop
# -----
def inference_loop():
    frame_count = 0
    while True:
        frame_count += 1

        # Replace this block with real camera read + model inference
        log.info(f"Frame {frame_count:06d} - running inference...")
        time.sleep(5) # simulates inference every 5 seconds

# -----
# Entry point
# -----
if __name__ == "__main__":

```

```
log.info("=" * 50)
log.info(" Edge AI Inference Service – Starting")
log.info("=" * 50)

run_startup_checks()
inference_loop()
```

Part 10 — The Complete `.service` File

```
# /etc/systemd/system/edgeai.service
# Edge AI Inference Service
# Analog Data – EdgeAI Engineering Bootcamp

[Unit]
Description=Edge AI Inference Service
After=network.target
StartLimitIntervalSec=60
StartLimitBurst=5

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/edgeai
EnvironmentFile=/home/pi/edgeai/.env
ExecStartPre=/bin/bash -c 'test -f /home/pi/edgeai/models/model.tflite'
ExecStart=/usr/bin/python3 /home/pi/edgeai/inference.py
ExecStop=/bin/kill -SIGTERM $MAINPID
Restart=always
RestartSec=5
TimeoutStopSec=10
StandardOutput=journal
StandardError=journal
SyslogIdentifier=edgeai

[Install]
WantedBy=multi-user.target
```

Part 11 — Full Setup Walkthrough (Step by Step)

Follow these steps in order the first time you set up a service.

Step 1 — Create your project folder

```
mkdir -p /home/pi/edgeai/models
```

Step 2 — Place your Python script

```
nano /home/pi/edgeai/inference.py
# (paste your script content and save)
```

Step 3 — Create and populate your `.env` file

```
nano /home/pi/edgeai/.env
# (add your KEY=value pairs and save)
```

Step 4 — Secure the `.env` file

```
chmod 600 /home/pi/edgeai/.env
```

Step 5 — Create the service file

```
sudo nano /etc/systemd/system/edgeai.service
# (paste the complete service file content and save)
```

Step 6 — Tell systemd about the new file

```
sudo systemctl daemon-reload
```

You must run `daemon-reload` every time you create or edit a service file. Without it, systemd is still reading the old version (or doesn't know the file exists yet).

Step 7 — Start the service

```
sudo systemctl start edgeai
```

Step 8 — Verify it is running

```
sudo systemctl status edgeai
```

What you want to see:

```
● edgeai.service - Edge AI Inference Service
   Loaded: loaded (/etc/systemd/system/edgeai.service; disabled)
   Active: active (running) since Thu 2026-07-17 18:30:00 IST; 3s ago
     Main PID: 2341 (python3)
        Tasks: 1 (limit: 4915)
      Memory: 47.3M
         CPU: 1.234s
       CGroup: /system.slice/edgeai.service
               └─2341 /usr/bin/python3 /home/pi/edgeai/inference.py

Jul 17 18:30:00 raspberrypi edgeai[2341]: Edge AI Inference Service - Starting
Jul 17 18:30:00 raspberrypi edgeai[2341]: All checks passed. Starting inference
```

```
loop.  
Jul 17 18:30:00 raspberrypi edgeai[2341]: Frame 000001 – running inference...
```

Key things to check:

- Active: active (running) → running correctly 
- Active: failed → something went wrong 
- Active: activating (start-pre) → running ExecStartPre , still starting

Step 9 — Watch the logs live

```
journalctl -u edgeai -f
```

Press `Ctrl + C` to stop watching.

Step 10 — Enable auto-start at boot

```
sudo systemctl enable edgeai
```

Output:

```
Created symlink /etc/systemd/system/multi-user.target.wants/edgeai.service  
→ /etc/systemd/system/edgeai.service
```

Step 11 — Reboot and confirm it works

```
sudo reboot
```

After the Pi restarts, SSH back in and check:

```
sudo systemctl status edgeai  
# Should show: Active: active (running)
```

If it is running without you doing anything — you have a working production service. 

Part 12 — `journalctl` — Reading Service Logs

`journalctl` is the tool for reading logs from the systemd journal — the central logging system where all your service's output goes.

Basic log commands

```
# View all logs for the edgeai service (oldest first)  
journalctl -u edgeai  
  
# Watch logs in real time – new lines appear as they are written  
journalctl -u edgeai -f
```

```
# Show the last 50 lines and then watch live
journalctl -u edgeai -n 50 -f

# Show only the last 100 lines
journalctl -u edgeai -n 100
```

Filtering by time

```
# Logs since a specific date and time
journalctl -u edgeai --since "2026-07-17 18:00:00"

# Logs from the last hour
journalctl -u edgeai --since "1 hour ago"

# Logs from today only
journalctl -u edgeai --since today

# Logs between two times
journalctl -u edgeai --since "18:00" --until "19:00"
```

Filtering by severity level

```
# Show only error messages and above
journalctl -u edgeai -p err

# Show warnings and above
journalctl -u edgeai -p warning

# Show all levels including debug
journalctl -u edgeai -p debug
```

Severity levels from lowest to highest: debug → info → notice → warning → err → crit → alert → emerg

Other useful options

```
# Show logs from the most recent boot only
journalctl -u edgeai -b

# Show logs from the previous boot (before the last reboot)
journalctl -u edgeai -b -1

# Output without line wrapping (good for copying log lines)
journalctl -u edgeai --no-pager

# Reverse order - newest logs at the top
journalctl -u edgeai -r

# Show timestamps in a readable format with microseconds
journalctl -u edgeai -o short-precise
```

Checking journal disk usage and cleaning old logs

```
# See how much disk space the journal is using
journalctl --disk-usage

# Remove logs older than 7 days
sudo journalctl --vacuum-time=7d

# Keep only the last 100MB of logs
sudo journalctl --vacuum-size=100M
```

Why this matters on a Raspberry Pi: The journal stores logs on the SD card. On a long-running Pi that generates lots of log output, the journal can grow to fill the SD card over time. Running `--vacuum-time=7d` periodically (via a cron job) keeps it under control.

Part 13 — Making Changes After Initial Setup

When you edit the Python script only

```
sudo systemctl restart edgeai
```

No `daemon-reload` needed — the service file did not change.

When you edit the `.service` file

```
sudo systemctl daemon-reload    # REQUIRED — reads the updated file
sudo systemctl restart edgeai
```

When you edit the `.env` file

```
sudo systemctl restart edgeai    # restart loads the new env values
```

When you want to view the service file as systemd sees it

```
systemctl cat edgeai
```

Part 14 — Troubleshooting a Failing Service

When a service fails to start, follow this process in order:

Step 1 — Check the status

```
sudo systemctl status edgeai
```

Read the `Active:` line and the last few log lines at the bottom of the output. Often the error is right there.

Step 2 — Read the full logs

```
journalctl -u edgeai -n 50 --no-pager
```

Look for lines labelled `ERROR` , `CRITICAL` , or Python tracebacks.

Step 3 — Test the script manually as the service user

```
sudo -u pi /usr/bin/python3 /home/pi/edgeai/inference.py
```

This runs the script exactly as systemd would (as user `pi`) but in your terminal, where you can see the error immediately. This is the fastest way to reproduce a startup failure.

Step 4 — Test with environment variables loaded

```
sudo -u pi bash -c 'set -a; source /home/pi/edgeai/.env; set +a; /usr/bin/python3 /home/pi/edgeai/inference.py'
```

This loads the `.env` file the same way `EnvironmentFile=` does, then runs the script.

Common Problems and Their Fixes

Error message	Likely cause	Fix
No such file or directory	Wrong path in <code>ExecStart</code> , <code>WorkingDirectory</code> , or <code>EnvironmentFile</code>	Double-check all paths are absolute and correct. Use <code>ls</code> to verify.
Permission denied	Wrong <code>User=</code> , or file is not readable by that user	Check file ownership with <code>ls -l</code> . Ensure <code>User=pi</code> owns the files.
ModuleNotFoundError: No module named 'x'	Python package not installed for the correct user	Install with <code>pip3 install x --break-system-packages</code> or use a venv
active (activating) forever	Script never finishes starting	Your script may be hanging at startup — add a log line at the very beginning to see how far it gets
Changes not taking effect	Forgot <code>daemon-reload</code> after editing the service file	Always run <code>sudo systemctl daemon-reload</code> after editing any <code>.service</code> file
Service not starting at boot	Service not enabled	Run <code>sudo systemctl enable edgeai</code>
<code>.env</code> values are empty	<code>EnvironmentFile=</code> path is wrong or file has wrong	Verify path with <code>ls -la /home/pi/edgeai/.env</code> , check

Error message	Likely cause	Fix
	permissions	permissions are 600
Service in failed state, won't restart	Hit StartLimitBurst crash limit	Fix the root cause, then run <code>sudo systemctl reset-failed edgeai</code> and <code>sudo systemctl start edgeai</code>
<code>ExecStartPre</code> failed	Pre-condition check failed (e.g. model file missing)	Check whether the model file exists at the path in <code>.env</code>

Part 15 — Quick Reference Card

Service file fields

Field	Section	Purpose
<code>Description=</code>	<code>[Unit]</code>	Human-readable service name
<code>After=</code>	<code>[Unit]</code>	Start only after these milestones
<code>StartLimitBurst=</code>	<code>[Unit]</code>	Max restarts before giving up
<code>Type=simple</code>	<code>[Service]</code>	Process type — use simple for Python
<code>User=</code>	<code>[Service]</code>	Run as this user account
<code>WorkingDirectory=</code>	<code>[Service]</code>	Set current directory before running
<code>EnvironmentFile=</code>	<code>[Service]</code>	Load variables from this <code>.env</code> file
<code>ExecStartPre=</code>	<code>[Service]</code>	Run this check before starting
<code>ExecStart=</code>	<code>[Service]</code>	The command that runs your program
<code>ExecStop=</code>	<code>[Service]</code>	Command to run on stop
<code>Restart=always</code>	<code>[Service]</code>	Auto-restart on any exit
<code>RestartSec=5</code>	<code>[Service]</code>	Wait 5 seconds before restarting
<code>TimeoutStopSec=</code>	<code>[Service]</code>	Wait this long for clean shutdown
<code>StandardOutput=journal</code>	<code>[Service]</code>	Send stdout to journal
<code>StandardError=journal</code>	<code>[Service]</code>	Send stderr to journal
<code>SyslogIdentifier=</code>	<code>[Service]</code>	Label in log output
<code>WantedBy=multi-user.target</code>	<code>[Install]</code>	Enable at normal boot

systemctl commands

Command	What it does
<code>sudo systemctl daemon-reload</code>	Tell systemd to re-read all service files
<code>sudo systemctl start edgeai</code>	Start the service now
<code>sudo systemctl stop edgeai</code>	Stop the service now
<code>sudo systemctl restart edgeai</code>	Stop then start
<code>sudo systemctl status edgeai</code>	Show current state and recent logs
<code>sudo systemctl enable edgeai</code>	Auto-start at every boot
<code>sudo systemctl disable edgeai</code>	Remove auto-start at boot
<code>systemctl cat edgeai</code>	View the service file as systemd reads it
<code>systemctl is-active edgeai</code>	Print active/inactive (useful in scripts)
<code>sudo systemctl reset-failed edgeai</code>	Clear failed state after fixing crash loop

journalctl commands

Command	What it does
<code>journalctl -u edgeai</code>	All logs for edgeai
<code>journalctl -u edgeai -f</code>	Watch logs live
<code>journalctl -u edgeai -n 50</code>	Last 50 lines
<code>journalctl -u edgeai -n 50 -f</code>	Last 50 lines then watch live
<code>journalctl -u edgeai -p err</code>	Errors only
<code>journalctl -u edgeai -b</code>	Logs from this boot only
<code>journalctl -u edgeai --since today</code>	Today's logs only
<code>journalctl -u edgeai --no-pager</code>	Output without scrolling (good for copy-paste)
<code>journalctl --disk-usage</code>	See how much space logs are using
<code>sudo journalctl --vacuum-time=7d</code>	Delete logs older than 7 days

Summary Checklist

Use this to confirm you have understood and practised every topic in this session.

- ☐ I can explain in one sentence what systemd does and why it is better than running scripts manually
- ☐ I can write a complete `.service` file from memory with all three sections: `[Unit]` , `[Service]` , `[Install]`
- ☐ I know what `Description=` and `After=network.target` do in `[Unit]`

- ☐ I know what `StartLimitIntervalSec=` and `StartLimitBurst=` protect against
 - ☐ I know what `Type=simple` , `User=` ,and `WorkingDirectory=` do in `[Service]`
 - ☐ I can explain what `ExecStartPre=` is and give a real example of when to use it
 - ☐ I know why `ExecStart=` always needs full absolute paths
 - ☐ I can explain the difference between `Restart=always` and `Restart=on-failure`
 - ☐ I know what `RestartSec=5` does and why a zero-second delay can be dangerous
 - ☐ I know what `StandardOutput=journal` and `SyslogIdentifier=` do
 - ☐ I can create a `.env` file, set `chmod 600` on it, and explain why that permission matters
 - ☐ I can add `EnvironmentFile=` to a service file and read the values in Python using `os.environ.get()`
 - ☐ I know the correct order of setup steps: write script → write `.env` → write service file → `daemon-reload` → `start` → `status` → `enable`
 - ☐ I understand the difference between `systemctl start` (run now) and `systemctl enable` (run at boot)
 - ☐ I can watch live service logs with `journalctl -u edgeai -f`
 - ☐ I can filter logs by time, severity level, and number of lines
 - ☐ I can follow the troubleshooting process: `status` → `journalctl` → manual test → fix → restart
 - ☐ I can name at least 5 entries from the common problems table and their fixes
-

Notes prepared by Analog Data — EdgeAI Engineering Bootcamp Copyright 2026 Analog Data. Strictly for enrolled students only.